Adventist University of Central Africa
P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

BIG DATA ANALYTICS | FINAL PROJECT

Distributed Multi-Model Analytics

for E-commerce Data with MongoDB, HBase and Apache Spark

Technical Report

Student: Odile Umwiza
Student ID: 101365
Course: Big Data Analytics
Submission: 11 November 2026

Project repository
https://github.com/umwizaodile/auca-bigdata-project

A reproducible analytics system that places e-commerce business entities in MongoDB, browsing sessions in HBase, and cross-store analytics in Apache Spark.

Executive Summary

This project builds and validates a multi-model analytics workflow over a synthetic e-commerce dataset: 10,000 users, 5,000 products in 25 categories, 200,000 browsing sessions and 50,000 transactions over 91 days. MongoDB stores transactions, products and user purchase summaries; HBase stores browsing sessions and product-day measures; Apache Spark provides shared data cleaning, recommendation indicators, SQL analysis, simulated streaming and integrated cross-store joins. The design matches each platform to its access pattern.MongoDB and HBase run in Docker containers and Spark runs locally, so the whole system is reproducible on one laptop..MongoDB and HBase run in Docker containers and Spark runs locally, so the whole system is reproducible on one laptop.

The integrated funnel converts 20.3% of sessions to recorded purchases. The largest loss occurs between cart and checkout: 44.9% of cart sessions continue. Abandoned-cart value is 17.9 million versus 7.9 million in purchased revenue. Regular buyers represent 52.7% of users and 66.3% of revenue. Comparisons with random baselines show that product co-viewing, category affinity and several channel, device and cohort differences are not reliable signals in this synthetic dataset. Revenue, streaming totals and funnel counts were reconciled across the workflows.

200,000	50,000	3	20.3%
browsing sessions	transactions	data platforms	session conversion

Contents

1 Introduction **3**

2 System Architecture and Technology Selection **3**

3 Data Modeling and Justification **4**

3.1 MongoDB . . . 4

3.2 HBase . . . 4

3.3 Why the Data Went to MongoDB or HBase . . . 4

4 Spark Processing Pipelines **5**

4.1 Cleaning and Data Quality . . . 5

4.2 Recommendation Indicators . . . 5

4.3 Spark SQL . . . 5

4.4 Simulated Stream Processing . . . 6

4.5 Optimizations Considered . . . 6

5 Integrated Analytics: Funnel Conversion **6**

6 Key Findings and Visualizations **7**

7 Methodology and Validation **8**

8 Scalability **8**

9 Limitations and Future Work **8**

10 Lessons Learned **8**

Repository and References **8**

1 Introduction

The project uses each technology where its data model and access pattern are a good fit. MongoDB handles business entities with nested relationships; HBase handles wide, ordered session and product time-series records; Apache Spark performs large scans, transformations and joins. The lecturer's generator produced about 400 MB of data. Its default of 2 million sessions and 500,000 transactions was reduced to 200,000 sessions and 50,000 transactions so the full system could run on an 8 GB laptop. The code is not tied to the reduced scale.

The implementation also maps the requested project requirements to repository code, making the analysis traceable to the submitted work.

Requirement	Report	Repository files
MongoDB schema, justification and aggregation pipelines	3.1	`load_mongodb.py`, `mongo_queries.py`, `mongo_session_queries.py`
HBase families, row keys, loading and session queries	3.2	`hbase_schema.hbase`, `load_hbase.py`, `hbase_queries.py`
Spark batch cleaning, recommendations and cohorts	4.1–4.2	`spark_clean.py`, `spark_recommend.py`, `spark_sql.py`
SQL, stream replay and integrated cross-store analysis	4.3–5	`spark_sql.py`, `spark_streaming_sim.py`, `spark_integrated.py`
Visualizations, scaling and limitations	6–8	`make_charts.py`, `charts/`, `README.md`

Table 1: Traceability from project requirements to report sections and source files.

2 System Architecture and Technology Selection

The generator's JSON data feeds three paths. MongoDB and HBase are loaded with records that suit their access patterns. Spark reads sessions from partitioned JSON Lines files and joins with data from both stores for the integrated analysis. MongoDB and HBase run in Docker containers, keeping setup and reloads reproducible.

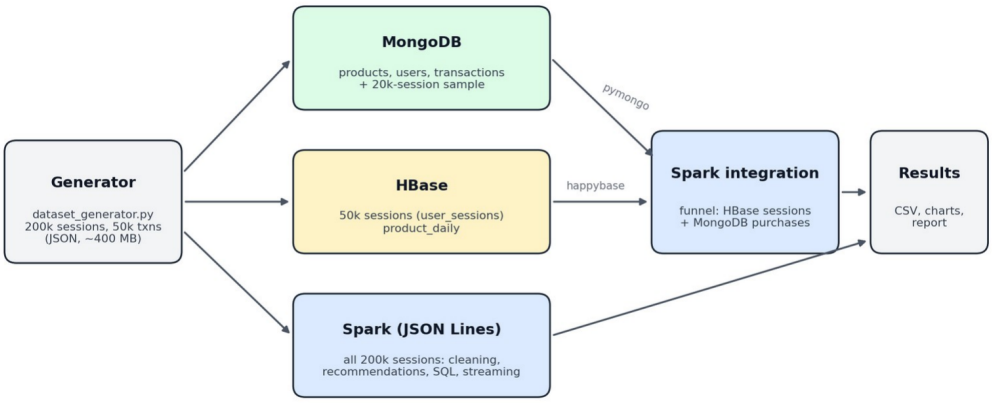


Figure 1: Architecture: data placement and how Spark joins the stores.

Analytical task	Technology	Why it fits
Revenue by category, customer segments and top products	MongoDB	Transactions embed items, while a purchase summary sits in each user document. Aggregations need few joins; indexes make lookups cheap.
Latest sessions of a user and product metrics over time	HBase	Sorted row keys place a user's sessions or a product's daily measures in a short prefix or range scan.
Cleaning, recommendations, SQL, stream replay and funnel joins	Apache Spark	Partitioned full scans and parallel joins support the same logic on one machine or a cluster.

Table 2: Platform selection by analytical workload.

3 Data Modeling and Justification

3.1 MongoDB

- **Transactions embed line items.** A line item has no meaning without its transaction and each basket is small and bounded. Each item keeps its own `unit_price`, preserving the price at purchase time if the catalog price changes later.
- **Products embed price history.** The category name is copied into each product as deliberate denormalization. Category revenue then needs no extra join, and category membership changes rarely.
- **Users carry a computed purchase summary.** Order count, total spend, first purchase and last purchase are calculated at load time, avoiding a rescan of 50,000 transactions during segmentation. Page views are not embedded in users: the list is unbounded and belongs in session records and HBase.
- Dates are stored as date values and indexes cover query fields. A one-user order lookup examined 4 documents with an index versus 50,000 without it. The generated schema has no `subcategory_id` on products and no user age, so customer segments use purchase frequency.

3.2 HBase

- `user_sessions` uses row key `user_id#reverse_timestamp#session_id`. The reversed, zero-padded timestamp groups one user's sessions newest first; starting with the user ID spreads writes, while the session ID suffix guarantees uniqueness.
- Three column families divide the data by access need: `info` holds session facts, `pv` stores one column per page view, and sparse `cart` stores cart events. Reading only `info` returned 52 cells instead of 94 for the same rows, 45% fewer.
- `product_daily` uses key `product_id#yyyymmdd` and family `m` for views, cart additions and purchases. A month of measures for one product is a single range scan.

The load contained 50,000 sessions and 142,040 product-day rows. Prefix, time-window and range scans took 8–19 ms; reading all page views of a session took 76–93 ms. Results were identical after restarting the containers.

3.3 Why the Data Went to MongoDB or HBase

Aspect	MongoDB	HBase
Data model	Rich nested documents.	Sparse, timestamped cells in column families.
Access pattern	Ad-hoc filters, grouping and joins such as `$lookup`.	Known-key prefix and range scans.
Data held	Transactions, products, users and a 20,000-session sample.	50,000 sessions and daily product metrics.
Measured strength	Indexed lookup examined 4 rather than 50,000 documents.	Family pruning returned 52 rather than 94 cells.
Trade-off	Unbounded arrays and unindexed scans are costly.	No secondary index or joins; the key must fit the query.

Table 3: Observed strengths and trade-offs of the two database models.

4 Spark Processing Pipelines

The raw files are JSON arrays, which Spark cannot split efficiently. They were converted to JSON Lines so records could be divided into partitions and processed in parallel. Jobs use explicit schemas instead of an inference pass; `cart_contents`, whose keys are product IDs, is read as a map. A shared cleaning module keeps quality flags and downstream metrics consistent.

4.1 Cleaning and Data Quality

Cleaning parses types, normalizes text, removes duplicates and adds flags rather than deleting anomalous rows. The following checks cover all 200,000 sessions and 50,000 transactions.

Check	Rows	Share
Duplicate or missing keys; line items pointing to unknown products	0	0%
Sessions visiting cart with an empty cart	43,590	21.8%
Sessions reaching checkout or confirmation with an empty cart	29,051	14.5%
Sessions marked converted without a transaction	15,515	7.8%
Transactions without a session link	24,906	49.8%
Users with `last_active` earlier than their last purchase	8,874	88.7%
Non-US country paired with a US state code	199,183	99.6%

Table 4: Data quality checks and flagged records.

Only 25,094 of the 40,609 converted sessions (61.8%) have a transaction record. The generator stops creating transactions at a 50,000 quota, and about half of those records are filler transactions without a session link. Later sessions therefore lack a purchase record. The integrated funnel uses the first 50,000 sessions, where purchases are complete, and cross-checks the rates against the full dataset.

4.2 Recommendation Indicators

Support, confidence and lift were computed for every co-viewed product pair from 200,000 sessions and 4,766 viewed products. Pair counts were compared with a Poisson random baseline because the generator selects products at random. Pairs observed three times numbered 2,060 against 1,893 expected; pairs observed four times numbered 68 against 49; pairs observed five times numbered 2 against 1. Only two pairs reached support of five, so there is no stable viewed-together signal. Lift values around 40 came from tiny expected counts of about 0.1. Category affinity from 50,000 baskets ranged from 0.85 to 1.17, within the approximately ± 0.19 band explained by random variation. The fallback recommender suggests best sellers from the same category.

4.3 Spark SQL

Five queries run on views named after the stores they represent (`mongo_*`, `hbase_*`), making cross-system join logic visible. Analyses include monthly revenue with a `LAG` growth window, top-three products per category using `RANK` and window sums, spend quartiles joined to session engagement with `NTILE`, inventory risk through a three-way join, and a registration-month cohort matrix. Total revenue is 53,926,975.67 in both MongoDB and Spark SQL. Revenue is otherwise about 600,000 per day; the apparent 79.8% August increase is an artifact of July containing only two days of data.

4.4 Simulated Stream Processing

The 200,000 sessions are replayed in time order as 91 daily micro-batches of about 2,200 sessions. For each batch, Spark computes metrics while the driver retains state across batches: cumulative and rolling seven-batch conversion, trending products over a three-batch window, and an alert when a batch falls outside a three-standard-deviation band. Final streaming totals (200,000 sessions, 40,609 conversions and 20.3%) exactly match the batch totals. One alert was raised on 8 September (23.0% versus 20.4%, 3.0 standard deviations). Across 91 batches the probability of at least one such crossing is about 22%, so the alert is ordinary variation. A production version could read from Kafka with Structured Streaming; deterministic replay keeps the analysis reproducible on a laptop.

4.5 Optimizations Considered

The pipeline uses JSON Lines for parallel reads, explicit schemas, and 8 shuffle partitions instead of Spark's default 200 on a single machine. It caches reused small tables, broadcasts small lookup tables, uses higher-order functions instead of exploding nested arrays, and aggregates quality flags in one pass. MongoDB applies `$limit` before `$lookup`; HBase reads only required column families; writes are batched in groups of 1,000 rows.

5 Integrated Analytics: Funnel Conversion

The business question is where customers are lost between viewing a product and recording a purchase, and whether the pattern changes by traffic source or device. The workflow reads session stages, cart columns, referrer and device from HBase; reads linked transactions from MongoDB with PyMongo (25,094 transactions); joins on `session_id` in Spark; and compares the result with the same funnel over all 200,000 Spark sessions.

Funnel stage	Sessions	All sessions	Continue from prior
Sessions	50,000	100%	n/a
Viewed a product	49,335	98.7%	98.7%
Added to cart	32,413	64.8%	65.7%
Reached checkout	14,540	29.1%	44.9%
Completed checkout	10,164	20.3%	69.9%
Purchase recorded in MongoDB	10,164	20.3%	100%

Table 5: Conversion funnel from 50,000 HBase sessions joined with MongoDB purchases.

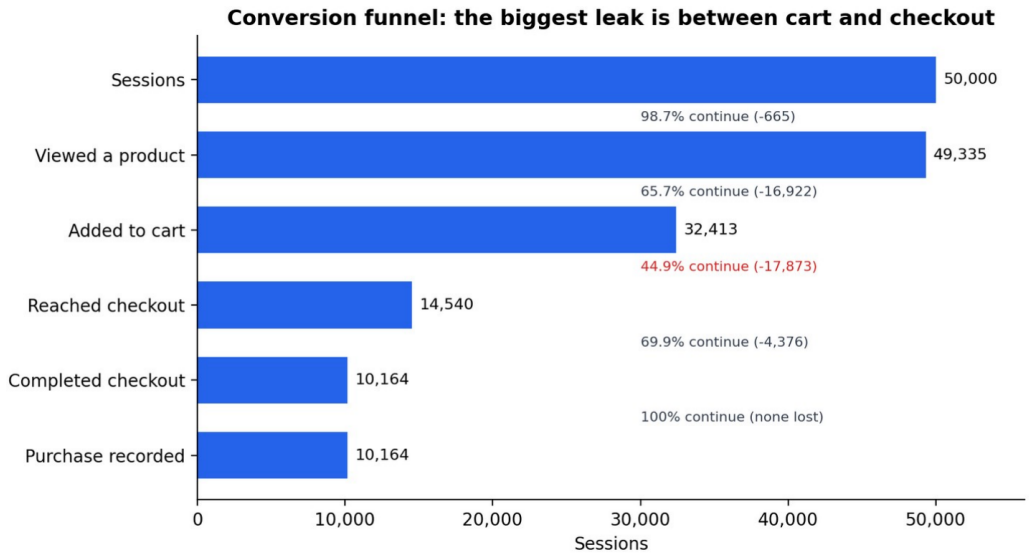


Figure 2: Conversion funnel: the largest loss is between cart and checkout.

Abandoned carts (22,249) plus completed ones (10,164) equal the 32,413 carts exactly. No completed session lacks a MongoDB transaction and no transaction belongs to an unfinished session. The first four step rates differ by at most 0.3 percentage points from the all-session funnel. HBase reads only required families and the small MongoDB input is broadcast; production connectors could replace the native clients.

6 Key Findings and Visualizations

1. **Cart-to-checkout is the largest leak.** Only 44.9% of carts reach checkout (17,873 sessions lost); a further 30% abandon at checkout. Test fewer steps, earlier shipping-cost disclosure and cart reminders.
2. **Abandoned carts have material value.** Cart value is 17.9 million versus 7.9 million of purchased revenue. A 10% recovery scenario would add about 22.5% of purchased revenue; this is a scenario, not a forecast, and cart value is before discounts.
3. **Regular buyers drive revenue.** Buyers with 5–9 orders are 52.7% of users and 66.3% of revenue. Loyal buyers (3.4% of users) spend 10.7 times as much as one-time buyers.
4. **Browsing is a weak spend proxy.** Average spend rises 4.5 times from the lowest to highest quartile, while sessions rise only 9%. Purchase history is a better customer-value signal than engagement.
5. **Restock high-demand items.** 163 products (3.3%) are out of stock; the highest-demand products are still active, viewed and added to carts. Prioritize replenishment and deactivate products at zero stock.
6. **Differences are not distinguishable from noise.** Categories earn within 1.44 times of one another, referrer conversion ranges from 19.4% to 21.2%, and device differences remain inside the noise band. Most-viewed products (26–27 views) match chance, so marketing budget shifts are not justified.

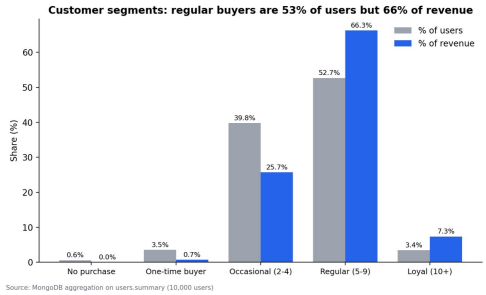


Figure 3: Customer segments: user share against revenue share.

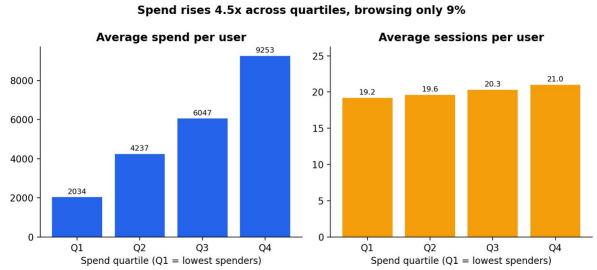


Figure 4: Spend quartiles compared with browsing engagement.

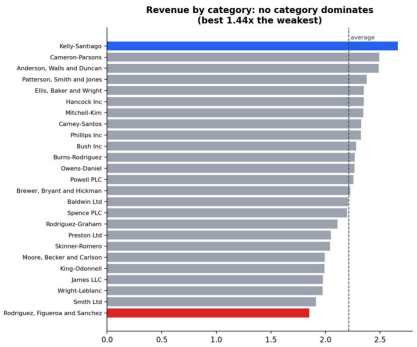


Figure 5: Revenue by category: the strongest category earns 1.44 times the weakest.

7 Methodology and Validation

Every reported pattern was tested against a random baseline or a noise band and reconciled across systems wherever possible.

Validation check	Result
MongoDB revenue against Spark SQL	53,926,975.67 in both systems.
Category revenue before discounts against user totals after discounts	Difference of 1,382,571 equals total discounts (2.5%).
Abandoned plus completed carts	22,249 + 10,164 = 32,413 carts.
HBase/MongoDB purchase reconciliation	Zero converted sessions without a transaction and zero transactions on unfinished sessions.
Funnel on 50,000 stored sessions against all 200,000 sessions	First four step rates within 0.3 percentage points.
Streaming totals against batch totals	200,000 sessions and 40,609 conversions, identical.
Co-viewed pairs against Poisson baseline	Observed counts close to chance (2,060 versus 1,893 for pairs seen three times).
Referrer and device differences	All inside the three-standard-deviation noise band.
Most-viewed products against simulated chance	Tenth place at 26 views; simulation result 26 (range 26–27).

Table 6: Cross-system and statistical validation results.

8 Scalability

- **Spark:** partitioned input and unchanged processing code can run on a cluster; the default 2-million-session dataset would be about 3.5 GB of JSON Lines.
- **HBase:** regions distribute writes; user-prefixed keys support targeted reads. Bulk processing could use a Spark-HBase connector.
- **MongoDB:** transactions can be sharded on `user_id`; indexes support lookups and user summaries avoid rescanning purchases.
- **Streaming:** replay logic maps to Kafka Structured Streaming with checkpoints and watermarks for late events.

9 Limitations and Future Work

- The data is synthetic and largely random; category affinity, cohorts and channels are weak by construction, and geographic fields and `last_active` are unreliable.
- The reduced dataset and 50,000-transaction quota leave later sessions without purchase records. The integrated funnel therefore uses an early, complete sample.
- Product-day metrics use 50,000 loaded sessions. The 20,000-session MongoDB sample overlaps the HBase subset, so some cross-checks are not independent.
- The environment is single-node and unauthenticated; streaming replays stored events, native clients stand in for Spark connectors, and Parquet output was not implemented on Windows due to Hadoop helper-file requirements.
- Future work includes real transactional data, Spark connectors, Kafka streaming, an ALS collaborative-filtering recommender, scheduled user-summary refresh and Parquet storage.

10 Lessons Learned

- Verify where data lands: a local MongoDB on port 27017 silently received the first load. Moving Docker MongoDB to port 27018 and querying the store itself prevented a repeat.
- Quantify inconsistencies instead of trusting status flags: the 7.8% converted-without-transaction rate exposed the generator's quota, and each analysis now states its population.
- Test against chance: random baselines prevented high lift values from sparse pairs from being presented as a recommendation insight.

Repository and References

[1] Umwiza, O. (2026). *AUCA Big Data Analytics project repository: source code, results and screenshots.* https://github.com/umwizaodile/auca-bigdata-project.

[2] Lecturer-provided synthetic e-commerce generator and dataset. Dataset scale, schema and known generation constraints are documented in this report.